

SENTIMENT ANALYSIS ON IMDB DATASET USING WORD2VEC AND DEEP LEARNING

Complete Project Documentation

Author: Shubham Soni (CS22B1053)

Institute: Indian Institute of Information Technology, Raichur

Supervisor: Dr. Natesha B V

Semester: 6th Semester Mini Project (2025-26)

1. INTRODUCTION

DEFINITION OF SENTIMENT ANALYSIS:

Sentiment analysis (also called opinion mining) is a Natural Language Processing (NLP) task that determines the emotional tone, opinion, or attitude expressed in text. It classifies text into categories like positive, negative, or neutral.

This project implements sentiment analysis on the IMDB Movie Reviews dataset, classifying 50,000 movie reviews as POSITIVE or NEGATIVE using Word2Vec word embeddings combined with multiple deep learning architectures.

We compare 5 models: CNN, LSTM, Bi-LSTM, 2-Layer Bi-LSTM, and CNN-LSTM Hybrid. The CNN model achieved the highest accuracy of 89.02%, demonstrating that local phrase patterns (captured by convolution) are the strongest indicator of sentiment in movie reviews.

REAL-WORLD APPLICATIONS OF SENTIMENT ANALYSIS:

- E-commerce: Analyzing thousands of product reviews automatically
- Social media: Tracking brand perception on Twitter/X
- Finance: Predicting stock movements from news sentiment
- Customer service: Auto-routing complaints vs positive feedback
- Politics: Gauging public opinion on policies
- Healthcare: Detecting depression indicators in patient text

2. PROBLEM STATEMENT

TASK: Given a movie review (free-form English text), automatically classify whether the reviewer's opinion is POSITIVE or NEGATIVE.

This is a BINARY TEXT CLASSIFICATION problem.

Examples:

"This movie was absolutely fantastic! Great acting." -> POSITIVE

"Terrible movie. Complete waste of time." -> NEGATIVE

"The acting was good but the plot made no sense." -> AMBIGUOUS (model decides)

CHALLENGES:

- Reviews contain informal language, slang, abbreviations
- Sarcasm: "Oh great, another terrible sequel" (looks positive, means negative)
- Varying lengths: 1 sentence to multiple paragraphs
- Negation: "not bad" = positive, but contains "bad"
- Mixed sentiment: "Great acting but terrible plot"
- Domain-specific vocabulary: movie/film industry terms

3. PROJECT OBJECTIVES

1. Build a complete text preprocessing pipeline for movie reviews
2. Use pre-trained Word2Vec embeddings (300D) as word representations
3. Implement 5 different deep learning architectures for comparison
4. Identify the best-performing model for binary sentiment classification

5. Achieve competitive accuracy on the standard IMDB benchmark
6. Understand strengths/weaknesses of each architecture for NLP tasks

4. DATASET

NAME: IMDB Movie Reviews Dataset

SOURCE: <https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews>

DETAILS:

Total reviews: 50,000
Positive reviews: 25,000 (50%)
Negative reviews: 25,000 (50%)
Perfectly balanced (no class imbalance issue)

SPLIT:

Training: 40,000 reviews (80%)
Testing: 10,000 reviews (20%)

WHY THIS DATASET:

- Standard benchmark in NLP research (allows comparison with published results)
- Large enough for deep learning (50K samples)
- Perfectly balanced (no need for class weighting strategies)
- Well-studied with published baselines
- Binary classification (clear task definition)
- Freely available on Kaggle

5. TEXT PREPROCESSING PIPELINE

Raw text cannot be directly fed to neural networks. The preprocessing pipeline converts messy, raw reviews into clean, standardized numerical sequences.

STEP 1: FIX CONTRACTIONS

Input: "I don't think this movie's worth watching. It isn't good."
Output: "I do not think this movie is worth watching. It is not good."
WHY: "don't" and "do not" should be treated identically. Without expansion, they'd be different tokens with different (or missing) Word2Vec vectors.
LIBRARY: contractions library in Python

STEP 2: REMOVE HTML TAGS

Input: "

The movie was terrible."
Output: "The movie was terrible."
WHY: IMDB data is scraped from web pages, containing HTML formatting tags. These carry zero sentiment information and must be removed.
METHOD: Regular expression: `re.sub(r'<.*?>', '', text)`

STEP 3: CONVERT TO LOWERCASE

Input: "ABSOLUTELY TERRIBLE Movie!!!"
Output: "absolutely terrible movie!!!"
WHY: "Movie" and "movie" should be the same word. Word2Vec has vectors for lowercase words. Without this, vocabulary doubles unnecessarily.

STEP 4: REMOVE SPECIAL CHARACTERS AND PUNCTUATION

Input: "Wow!!! 10/10 movie... absolutely amazing!!!"
Output: "Wow 10 10 movie absolutely amazing"
WHY: Punctuation and special characters don't have Word2Vec embeddings. They'd become OOV (out-of-vocabulary) and mapped to zero vectors anyway.
METHOD: `re.sub(r'[^\a-zA-Z\s]', '', text)`

STEP 5: REMOVE STOPWORDS

Input: "this is a very good movie and the acting was great"
Output: "good movie acting great"
WHY: Words like "is", "a", "the", "and" appear equally in positive AND

negative reviews. They don't carry sentiment information but add sequence length (more padding, slower processing).

LIBRARY: NLTK stopwords list (English, ~179 words)

CAUTION: Some implementations remove "not" - we keep it because it's crucial for sentiment ("not good" vs "good").

STEP 6: LEMMATIZATION

Input: "running movies loved better watching liked"

Output: "run movie love good watch like"

WHY: Reduces words to base form. "liked", "liking", "likes" all become "like". This (a) reduces vocabulary size, (b) ensures Word2Vec coverage (Word2Vec likely has "like" but maybe not "liking"), (c) groups semantically identical words.

LIBRARY: NLTK WordNetLemmatizer

vs. STEMMING: Stemming uses rules to chop endings ("studies" -> "studi", not a real word). Lemmatization uses dictionary ("studies" -> "study", valid). Lemmatization is better for Word2Vec because it produces actual words.

STEP 7: TOKENIZE AND PAD/TRUNCATE TO 300 TOKENS

- Convert text to sequence of integer indices (word -> number)

- Pad short sequences with zeros to reach 300

- Truncate long sequences to first 300 tokens

WHY 300: Covers majority of reviews. Neural networks need fixed-length input.

PREPROCESSING RESULT:

- Clean sequences of 300 integer tokens each
- Ready for Word2Vec embedding lookup
- Vocabulary reduced significantly (fewer unique words)

6. WORD EMBEDDINGS: WORD2VEC

DEFINITION OF WORD EMBEDDINGS:

Dense numerical vector representations of words where semantically similar words have similar vectors. Unlike one-hot encoding (sparse, 50000D, no meaning), embeddings are dense (300D) and capture semantic relationships.

DEFINITION OF WORD2VEC:

A neural network algorithm by Google (Mikolov et al., 2013) that learns word embeddings by predicting words from context. Core idea: words that appear in similar contexts have similar meanings.

Two training approaches:

- Skip-gram: Given center word, predict context words
- CBOW: Given context words, predict center word

PRE-TRAINED MODEL USED IN THIS PROJECT:

Name: Google News Word2Vec

Dimensions: 300 per word

Vocabulary: ~3 million words and phrases

Trained on: ~100 billion words from Google News

Library for loading: Gensim (KeyedVectors.load_word2vec_format)

WHY PRE-TRAINED (TRANSFER LEARNING):

- Trained on 100 BILLION words vs our 50K reviews (~10M words)
- Has seen millions of contexts for each word -> rich representations
- Words appearing only 2-3 times in our data would have poor self-trained vectors
- Pre-trained model already knows "good" is similar to "great", "terrible" to "horrible" - we don't need to re-learn this from scratch

EMBEDDING MATRIX CONSTRUCTION:

1. Build vocabulary from all preprocessed reviews (unique words)
2. Create matrix of shape: vocabulary_size x 300

3. For each word in vocabulary:
 - If found in Word2Vec: fill row with its 300D vector
 - If NOT found (OOV): fill row with zeros
4. Load this matrix as the Embedding layer's weights
5. Set trainable=False (freeze the layer)

WHY FREEZE THE EMBEDDING LAYER:

- Pre-trained vectors are high quality (from 100B words)
- Fine-tuning on just 40K reviews might damage general knowledge
- Reduces trainable parameters (prevents overfitting)
- Faster training (fewer gradients to compute)

HANDLING OOV (OUT-OF-VOCABULARY) WORDS:

- Movie slang, misspellings, character names may not exist in Google News vocab
- These get zero vectors (model effectively ignores them)
- Limitation: some domain-specific words carry sentiment but are lost

WHY WORD2VEC OVER ALTERNATIVES:

vs. One-Hot Encoding:

- One-hot: sparse, 500000D, no semantic similarity
- Word2Vec: dense, 300D, captures meaning
- "good" and "great" are far apart in one-hot, close in Word2Vec

vs. TF-IDF:

- TF-IDF gives one vector per DOCUMENT (bag-of-words style)
- Word2Vec gives per-word vectors preserving word ORDER
- Can't use TF-IDF with LSTM/CNN that need word sequences

vs. GloVe:

- Similar quality to Word2Vec, different training algorithm
- GloVe uses word co-occurrence matrix + factorization
- Either would work; Word2Vec chosen as the standard

vs. FastText:

- Handles OOV better (uses character n-grams within words)
- "plot" -> uses "plo" + "t" subwords
- Would be slightly better for informal text
- Word2Vec is simpler and standard

vs. BERT Embeddings:

- BERT: contextual (different vector for same word in different context)
- Word2Vec: static (same vector always)
- BERT much more powerful but 110M+ parameters
- Way beyond scope for 6th-sem project

PROS OF WORD2VEC:

- + Captures semantic meaning
- + 300D is efficient (vs 500000D one-hot)
- + Pre-trained on massive corpus (transfer learning)
- + Fast lookup (just matrix indexing)
- + Works well with sequential models

CONS OF WORD2VEC:

- Static (can't distinguish "bank" river vs financial)
- OOV words get zero vectors
- Pre-trained on news (may miss movie-specific slang)
- Doesn't capture word order by itself

7. MODEL ARCHITECTURES

Common pipeline for all models:

Input(300 tokens) -> Embedding(300D, frozen Word2Vec) -> [Model] -> Dense -> Sigmoid -> Output

7.1 CNN (Convolutional Neural Network) - 89.02% (BEST)

DEFINITION: CNN applies learnable sliding filters over input to detect local patterns. For text, filters detect meaningful phrases (n-gram-like patterns).

HOW CNN WORKS FOR TEXT:

- Input: 300 words x 300D embedding = 300x300 matrix
- Conv1D filters of size k slide over k consecutive word vectors
- Filter size 3: looks at trigrams ("waste of time", "really loved it")
- Multiple filters (e.g., 128) = 128 different pattern detectors
- Each filter produces an activation map showing where patterns are found
- Max pooling: picks the STRONGEST activation (does this pattern exist ANYWHERE?)
- Result: fixed-length vector of detected pattern strengths

WHY CNN IS BEST FOR SENTIMENT:

Sentiment is mainly determined by LOCAL PHRASES:

- "waste of time" -> negative (3-word pattern)
- "absolutely fantastic" -> positive (2-word pattern)
- "worst movie ever" -> negative (3-word pattern)

CNN's filters are DESIGNED to capture exactly these patterns. It processes the entire sequence in PARALLEL (fast) and doesn't suffer from "forgetting" like LSTM does on long sequences.

ARCHITECTURE:

Embedding(300D) -> Conv1D(128 filters, size 3-5) -> MaxPool1D ->
Dropout(0.5) -> Dense(64, relu) -> Dense(1, sigmoid)

PROS: Fast (parallel), captures local patterns well, simple architecture

CONS: Can't capture long-range dependencies (word 10 relating to word 290)

7.2 LSTM (Long Short-Term Memory) - 88.05%

DEFINITION: LSTM is a Recurrent Neural Network variant with gating mechanisms that can learn long-range dependencies in sequential data.

HOW LSTM WORKS:

- Processes words one at a time, left to right
- Maintains CELL STATE (long-term memory) and HIDDEN STATE (short-term)
- Three gates at each step:
 - * FORGET gate: what to erase from memory (sigmoid * cell state)
 - * INPUT gate: what new info to store (sigmoid * tanh)
 - * OUTPUT gate: what to output (sigmoid * tanh of cell state)
- Final hidden state (after word 300) summarizes the entire review

WHY LSTM FOR SENTIMENT:

- Captures sequential dependencies ("not" modifying "good" later)
- Handles variable-length information (long reviews)
- Can remember important early words through gating mechanism

ARCHITECTURE:

Embedding(300D) -> LSTM(128 units) -> Dropout(0.5) -> Dense(1, sigmoid)

PROS: Handles sequences naturally, captures dependencies, handles negation

CONS: Sequential (slow, can't parallelize), may still forget very early info

7.3 Bi-LSTM (Bidirectional LSTM) - 87.63%

DEFINITION: Two LSTMs running simultaneously - one forward (left-to-right), one backward (right-to-left). Outputs concatenated.

HOW IT DIFFERS:

- Forward: "The movie was not good" -> processes left to right
- Backward: "good not was movie The" -> processes right to left
- Each word gets context from BOTH past AND future words
- Output dimension is 2x regular LSTM (forward + backward concatenated)

WHY IT DIDN'T BEAT LSTM (87.63% vs 88.05%):

- Double parameters -> higher overfitting risk on 40K samples
- For sentiment, forward-only reading usually sufficient
- Backward context less useful for opinion classification
- Would need more data/regularization to show benefit

ARCHITECTURE:

Embedding(300D) -> Bidirectional(LSTM(128)) -> Dropout(0.5) -> Dense(1, sigmoid)

7.4 2-Layer Bi-LSTM - 88.52%

DEFINITION: Two Bi-LSTM layers stacked. First layer outputs a sequence (not just final state), second layer processes that sequence.

HIERARCHY:

- Layer 1: Captures word-level and phrase-level patterns
- Layer 2: Captures sentence-level and review-level patterns
- Deeper = more abstract feature extraction

RESULT: 88.52% - better than single Bi-LSTM, depth helps.

ARCHITECTURE:

Embedding(300D) -> Bidirectional(LSTM(128, return_sequences=True)) ->
Bidirectional(LSTM(64)) -> Dropout(0.5) -> Dense(1, sigmoid)

7.5 CNN-LSTM Hybrid - 88.30%

DEFINITION: CNN extracts local features first, then LSTM processes them sequentially to understand the flow of detected patterns.

INTUITION:

- CNN says: "Here are the important phrases in this review"
- LSTM says: "How do these phrases connect and flow together?"
- Combines local feature detection with sequential understanding

WHY IT DIDN'T BEAT PURE CNN:

For binary sentiment, just DETECTING the patterns is enough. Understanding their SEQUENCE adds complexity/overfitting without proportional benefit. "waste of time" anywhere in the review -> negative, regardless of its sequential position.

ARCHITECTURE:

Embedding(300D) -> Conv1D(64) -> MaxPool1D -> LSTM(64) ->
Dropout(0.5) -> Dense(1, sigmoid)

8. WHY THESE MODELS?

WHY DEEP LEARNING OVER TRADITIONAL ML:

Approach	Accuracy	Why Not Chosen as Primary
Naive Bayes + BoW	~85%	Can't capture word order/semantics
SVM + TF-IDF	~88%	Needs manual feature engineering
Random Forest	~82-85%	Bad with sequential data
Word2Vec + DL	89.02%	CHOSEN - learns features automatically

| BERT (fine-tuned) | ~93-95% | Beyond project scope |

WHY 5 MODELS COMPARED:

The project's academic value comes from the COMPARISON - understanding WHY CNN beats LSTM for sentiment helps build intuition about NLP architectures. Just using one model would be a weaker contribution.

9. TRAINING SETUP

FRAMEWORK: TensorFlow / Keras

HARDWARE: Google Colab (free GPU - NVIDIA Tesla T4)

COMMON SETTINGS:

- Optimizer: Adam (adaptive learning rate, lr=0.001 default)
- Loss: Binary Crossentropy
- Output: Sigmoid (probability 0-1)
- Embedding: Word2Vec 300D (frozen)
- Input length: 300 tokens
- Split: 80/20 (40K train / 10K test)
- Regularization: Dropout (0.3-0.5)
- Batch size: 64 or 128
- Epochs: Early stopping on validation loss

EVALUATION METRICS:

- Accuracy: % correct predictions
- ROC-AUC: Discrimination ability (threshold-independent)
- Confusion Matrix: True/False Positive/Negative counts

10. RESULTS

Model	Accuracy	AUC
CNN	89.02%	0.96
2-Layer Bi-LSTM	88.52%	0.95
CNN-LSTM Hybrid	88.30%	0.95
LSTM	88.05%	0.95
Bi-LSTM	87.63%	0.95

KEY OBSERVATIONS:

1. CNN BEST (89.02%): Local phrase patterns drive sentiment
2. All >87%: Word2Vec provides strong foundation regardless of architecture
3. 2-Layer > 1-Layer Bi-LSTM: Depth helps
4. Hybrid < Pure CNN: Sequential modeling of features doesn't help binary task
5. AUC 0.95-0.96: All models discriminate well
6. 1.4% spread: Embeddings matter more than architecture

11. PROJECT STRUCTURE (FILES AND FOLDERS)

```
sentiment-analysis-on-IMDB-using-word2vec-and-deep-learning-technique/
|
+-- README.md
|    Project overview, results, setup instructions.
|
+-- Report_sentiment_Analysis.pdf
|    Complete academic report with methodology and results.
|
+-- dataset_info__text_processing.ipynb
```

```

| Jupyter notebook: dataset loading, exploration (shape, class
| distribution, review length statistics), and full preprocessing
| pipeline with step-by-step output visualization.
|
+-- output-of-text-processing.txt
| Shows sample reviews before and after each preprocessing step.
| Useful for verifying the pipeline works correctly.
|
+-- dataset/
| Contains the IMDB CSV file (review text + sentiment label).
| 50,000 rows, 2 columns (review, sentiment).
|
+-- text_processing/
| Complete preprocessing code:
| - Contraction expansion
| - HTML tag removal (regex)
| - Lowercase conversion
| - Special character removal
| - NLTK stopwords removal
| - NLTK lemmatization
| - Tokenization and sequence padding (Keras pad_sequences)
|
+-- word2vec/
| Embedding code:
| - Loading Google News Word2Vec model via Gensim
| - Building vocabulary from preprocessed reviews
| - Creating embedding matrix (vocab_size x 300)
| - Mapping words to vectors (OOV -> zero vector)
| - Saving embedding matrix for model training
|
+-- model_training/
| Training notebooks for all 5 architectures:
| - CNN: Conv1D + MaxPool + Dense (89.02%)
| - LSTM: LSTM layer + Dense (88.05%)
| - Bi-LSTM: Bidirectional LSTM + Dense (87.63%)
| - 2-Layer Bi-LSTM: Stacked Bidirectional (88.52%)
| - CNN-LSTM: Conv1D + MaxPool + LSTM + Dense (88.30%)
| Each includes model definition, compilation, training with
| callbacks, evaluation on test set.
|
+-- prediction/
| Inference code for classifying new reviews:
| - Load saved model
| - Apply same preprocessing pipeline
| - Generate embedding sequence
| - Run model prediction
| - Output: positive/negative with confidence
|
+-- final_output/
| Results and visualizations:
| - Confusion matrix for each model
| - ROC curves (AUC values)
| - Accuracy comparison bar chart
| - Training/validation loss curves

```

12. TECHNOLOGIES USED

- Python 3.x: Programming language
- TensorFlow/Keras: Deep learning model building and training
- Gensim: Loading pre-trained Word2Vec (KeyedVectors)
- NLTK: Stopwords, WordNetLemmatizer, tokenization
- NumPy: Array operations, sequence padding
- Pandas: DataFrame operations, CSV loading
- Matplotlib/Seaborn: Visualizations (plots, heatmaps)
- Google Colab: Free GPU (NVIDIA T4) for training

- Jupyter Notebook: Interactive development
- Regular Expressions (re): Text cleaning patterns

13. KEY DEFINITIONS

SENTIMENT ANALYSIS: Computational study of opinions expressed in text.

BINARY CLASSIFICATION: Task with only two possible output classes.

WORD EMBEDDING: Dense vector representation of a word capturing its meaning.

WORD2VEC: Algorithm that learns word embeddings from word context patterns.

CNN: Neural network using convolutional filters to detect local patterns.

RNN: Neural network that processes sequential data with hidden state memory.

LSTM: RNN variant with gates (forget/input/output) to handle long sequences.

BIDIRECTIONAL: Processing sequence in both forward and backward directions.

DROPOUT: Regularization that randomly disables neurons during training.

MAX POOLING: Taking the maximum value from a region (strongest signal).

SIGMOID: Activation function mapping values to [0,1] for binary probability.

BINARY CROSSENTROPY: Loss function measuring prediction error for binary tasks.

ADAM: Adaptive optimizer combining Momentum and RMSProp.

ROC-AUC: Area under True Positive Rate vs False Positive Rate curve.

TRANSFER LEARNING: Using knowledge from one task to help another task.

LEMMATIZATION: Reducing words to dictionary base form.

STOPWORDS: Common words (the, is, a) that don't carry task-specific meaning.

14. LIMITATIONS

1. Static embeddings: Word2Vec can't distinguish polysemy ("bank" meanings)
2. OOV words lost: Slang, misspellings get zero vectors
3. Binary only: No neutral/mixed class
4. English only: Preprocessing and embeddings are English-specific
5. Max 300 tokens: Long reviews truncated (potentially losing info)
6. No sarcasm handling: "What a masterpiece" (sarcastic) misclassified
7. Domain-specific: Trained on movies, may not generalize to other domains

15. FUTURE SCOPE

1. BERT/RoBERTa for 93-95% accuracy
2. Attention mechanism for interpretability
3. Multi-class sentiment (5 levels)
4. Multi-domain: products, tweets, news
5. Real-time streaming sentiment analysis
6. Multilingual support
7. Web deployment (Flask/Streamlit)
8. Sarcasm detection module

16. CONCLUSION

=====

This project demonstrates effective combination of pre-trained Word2Vec embeddings with deep learning for sentiment analysis. Key findings:

1. CNN achieves best accuracy (89.02%) - local phrases drive sentiment
2. All models >87% - Word2Vec embeddings provide strong semantic foundation
3. Architecture choice matters less than embedding quality (1.4% spread)
4. For binary sentiment, local pattern detection (CNN) > sequential modeling (LSTM)
5. Transfer learning (pre-trained Word2Vec) enables competitive results on 50K samples

The project provides a comprehensive comparison of deep learning approaches for NLP, demonstrating proficiency in text preprocessing, word embeddings, and neural network architecture design and evaluation.

=====

END OF DOCUMENTATION

=====